Software Architecture and Design Specification

Project:Healthcare Appointment & Patient Record System

Version: 1.0

Authors: Sairam Charan N — PES1UG24AM240

Sanketh Shivaji R — PES1UG24AM251

Santhosh M — PES1UG24AM252

Sumukh D Hegde — PES1UG24AM293

Date: 11-10-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	12-10-2026	Project Team	Initial SAD document

Approvals

Role	Name	Signature/Date
Course Coordinator	To be filled	To be filled

1. Introduction

1.1 Purpose

This document describes the architecture and design of the Healthcare Appointment & Patient Record System.

1.2 Scope

The system manages patients, doctors, appointments, medical records, and prescriptions.

1.3 Audience

Developers, QA engineers, project evaluators, and maintenance teams.

1.4 Definitions

Term	Meaning
SAD	Software Architecture and Design
SRS	Software Requirements Specification
UML	Unified Modeling Language
API	Application Programming Interface
RBAC	Role-Based Access Control
TLS	Transport Layer Security
RTM	Requirements Traceability Matrix

2. Document Overview

2.1 How to Use This Document

This document describes the system architecture, UML diagrams, design decisions, security measures, and API design.

2.2 Related Documents

- Software Requirements Specification (SRS)
- Software Test Plan (STP)
- Requirements Traceability Matrix (RTM)

3. Architecture

3.1 Goals & Constraints

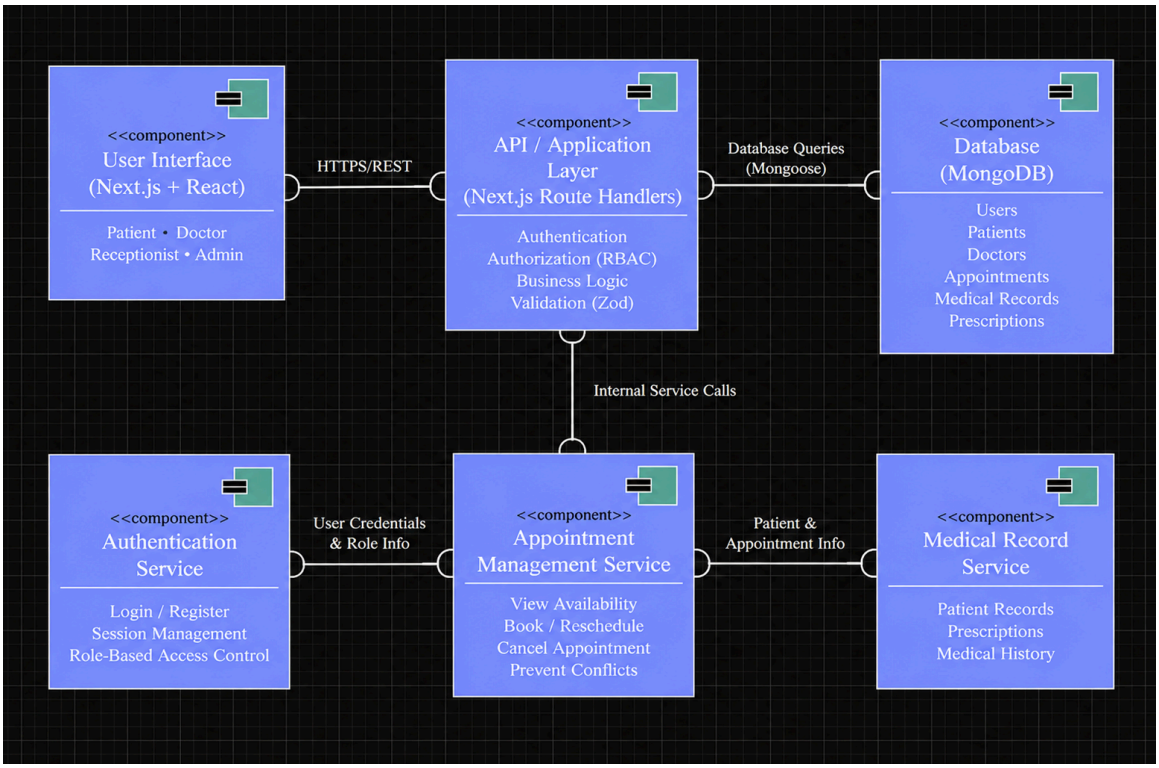
Goals: Secure, reliable, maintainable, and easy-to-use healthcare management system.

Constraints: Patient data must be protected, access must be role-based, and appointment conflicts must be prevented.

3.2 Stakeholders & Concerns

Stakeholder	Main Concern
Patients	Privacy and easy appointment booking
Doctors	Access to appointments and medical records
Receptionists	Patient registration and appointment management
Administrators	User management and system monitoring
Developers	Maintainability and modularity

3.3 Component (UML) Diagram



3.4 Component Descriptions

- **Next.js:** Provides application routing, pages, and server-side functionality.
- **Tailwind CSS:** Styles the user interface.
- **Route Handlers:** Process API requests and application logic.
- **Authentication & RBAC:** Verify users and restrict access according to their roles.
- **Mongoose:** Defines data models and communicates with MongoDB.
- **MongoDB:** Stores user, appointment, medical-record, and prescription data.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture is chosen to separate the presentation, application logic, and database layers. Next.js supports both frontend pages and backend API endpoints, reducing the need for a separate Express server. This structure improves maintainability and simplifies development.

3.6 Technology Stack & Data Stores

- **Frontend:** Next.js and React
- **Styling:** Tailwind CSS
- **Backend:** Next.js Route Handlers
- **Database:** MongoDB
- **ODM:** Mongoose
- **Validation:** Zod
- **Authentication:** Secure session-based authentication
- **Communication:** REST APIs over HTTPS with TLS 1.2 or higher
- **Version Control:** Git and GitHub

3.7 Risks & Mitigations

Risk	Mitigation
Unauthorized access	Authentication and role-based access control
Appointment conflicts	Validate availability before confirming bookings

Data loss	Regular database backups
Invalid input	Server-side validation using Zod
Exposure of medical data	HTTPS, secure sessions, and access restrictions

3.8 Traceability to Requirements

- **HCS-F-001 — Login:** Authentication module.
- **HCS-F-008 — Book Appointment:** Appointment module.
- **HCS-F-009 — Prevent Conflicts:** Appointment validation logic.
- **HCS-F-013 — Medical History:** Medical-record module.
- **HCS-SR-002 — Role-Based Access:** Authorization module.

3.9 Security Architecture

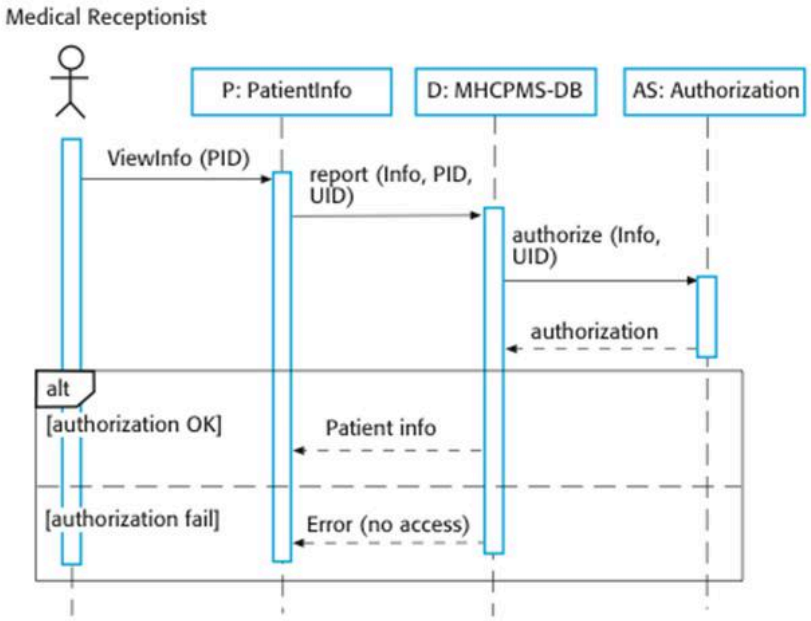
- **Authentication:** Verify registered email and password.
- **Authorization:** Restrict functions and records according to user roles.
- **Data Protection:** Use HTTPS with TLS 1.2 or higher.
- **Password Security:** Store passwords using secure one-way hashing.
- **Session Security:** Protect sessions and terminate them on logout.
- **Audit Logging:** Record important appointment and medical-record changes.

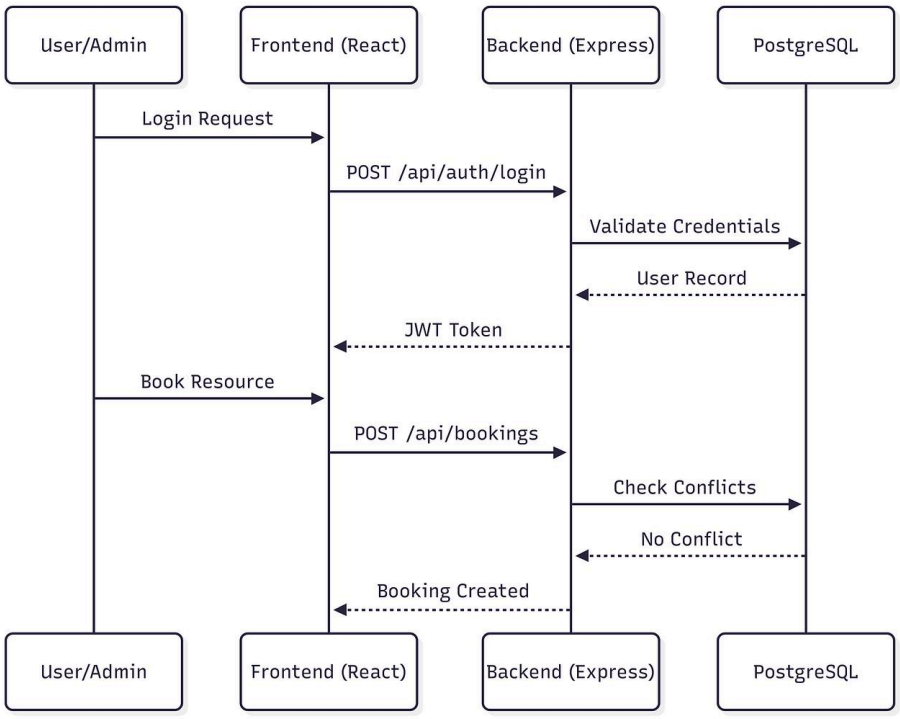
4. Design

4.1 Design Overview

The system follows a layered design using Next.js, Route Handlers, and MongoDB. The frontend handles user interactions, the application layer processes requests and business logic, and the database stores system information.

4.2 UML Sequence Diagrams





4.3 API Design

Endpoint	Method	Purpose
`/api/auth/login`	POST	Authenticate a user
`/api/appointments`	GET	View appointments permitted to the user
`/api/appointments`	POST	Book an appointment
`/api/appointments/[id]`	PATCH	Reschedule or update an appointment
`/api/appointments/[id]`	DELETE	Cancel an appointment
`/api/medical-records`	GET	Retrieve authorized medical records
`/api/prescriptions`	GET	View authorized prescriptions

These are proposed REST API routes for the design document; implement them in Next.js Route Handlers.

4.4 Error Handling, Logging & Monitoring

- Return clear error messages for invalid requests and failed operations.
- Use appropriate HTTP status codes for authentication, authorization, and appointment conflicts.
- Do not store passwords or sensitive medical information in application logs.
- Log important appointment and medical-record changes.
- Monitor server errors and failed requests.

4.5 UX Design

The interface shall provide simple navigation, readable text, and role-specific dashboards for patients, doctors, receptionists, and administrators. Forms shall display clear validation messages, and appointment availability shall be easy to understand.

4.6 Open Issues & Next Steps

- Complete the UML sequence diagrams.
- Implement authentication and role-based access control.
- Implement appointment booking and conflict prevention.
- Integrate MongoDB using Mongoose.
- Test API endpoints, data access, and error handling.
- Consider reminders and reporting as future enhancements.

5. Appendices

5.1 Glossary: SAD – Software Architecture and Design; SRS – Software Requirements Specification; UML – Unified Modeling Language; API – Application Programming Interface; RBAC – Role-Based Access Control; TLS – Transport Layer Security; ODM – Object Data Modeling; RTM – Requirements Traceability Matrix.

5.2 References: Next.js Documentation – https://nextjs.org/docs; React Documentation – https://react.dev; MongoDB Documentation – https://www.mongodb.com/docs; Mongoose Documentation – https://mongoosejs.com/docs; Tailwind CSS Documentation – https://tailwindcss.com/docs.

5.3 Tools: Visual Studio Code – Development; draw.io – UML diagrams; Git and GitHub – Version control; Postman – API testing; MongoDB Compass – Database inspection.